

## ЛЕКЦІЯ 5

### UNIFIED MODELING LANGUAGE +AI

**1.ПРОКЛАД UML-ДІАГРАМИ**

**2. ВИДИ UML-ДІАГРАМ**

**3. ЗАГАЛЬНА ХАРАКТЕРИСТИКА UML-ДІАГРАМ**

**4. ДІАГРАМА КЛАСІВ**

**5. ДІАГРАМА АКТИВНОСТІ**

**6. ДІАГРАМА ВИПАДКІВ ВИКОРИСТАННЯ (USE CASE DIAGRAM)**

**7. ДІАГРАМА ПОСЛІДОВНОСТІ (SEQUENCE DIAGRAM)**

**8. ДІАГРАМА РОЗГОРТАННЯ (Deployment Diagram)**

**9. АНАЛІЗ ЛІТЕРАТУРНИХ ДЖЕРЕЛ**

**10. UML та AI**

***1.ПРОКЛАД UML-ДІАГРАМИ***

**[MS LEC6-3]**

# представляє UML-діаграму університету

```
from graphviz import Digraph
```

```

def generate_university_uml():
    dot = Digraph("University UML", format="png")
    dot.attr(bgcolor="white")

    # Стилль для класів
    default_style = {
        "shape": "record",
        "style": "filled",
        "fontname": "Arial",
        "fontsize": "12"
    }

    # Додаємо класи
    dot.node("University", '''{
        University |
        - name: str\\l
        - location: str\\l|
        + getFaculties()\\l
        + getLibrary()\\l
    }''', fillcolor="lightblue", **default_style)

    dot.node("Faculty", '''{
        Faculty |
        - name: str\\l|
        + getDepartments()\\l
    }''', fillcolor="lightgreen", **default_style)

    dot.node("Department", '''{
        Department |
        - name: str\\l|
        + getProfessors()\\l
        + getCourses()\\l
    }''', fillcolor="lightcoral", **default_style)

    dot.node("Professor", '''{
        Professor |
        - name: str\\l
        - title: str\\l|
        + teach()\\l
        + evaluate()\\l
    }''', fillcolor="khaki", **default_style)

```

```

dot.node("Student", '''{
    Student |
    - name: str\\l
    - student_id: int\\l|
    + study()\\l
    + takeExam()\\l
}''', fillcolor="lightyellow",
**default_style)

dot.node("Course", '''{
    Course |
    - name: str\\l
    - credits: int\\l|
    + enrollStudent()\\l
    + assignProfessor()\\l
}''', fillcolor="lightpink", **default_style)

dot.node("Library", '''{
    Library |
    - books: int\\l|
    + borrowBook()\\l
    + returnBook()\\l
}''', fillcolor="lightcyan", **default_style)

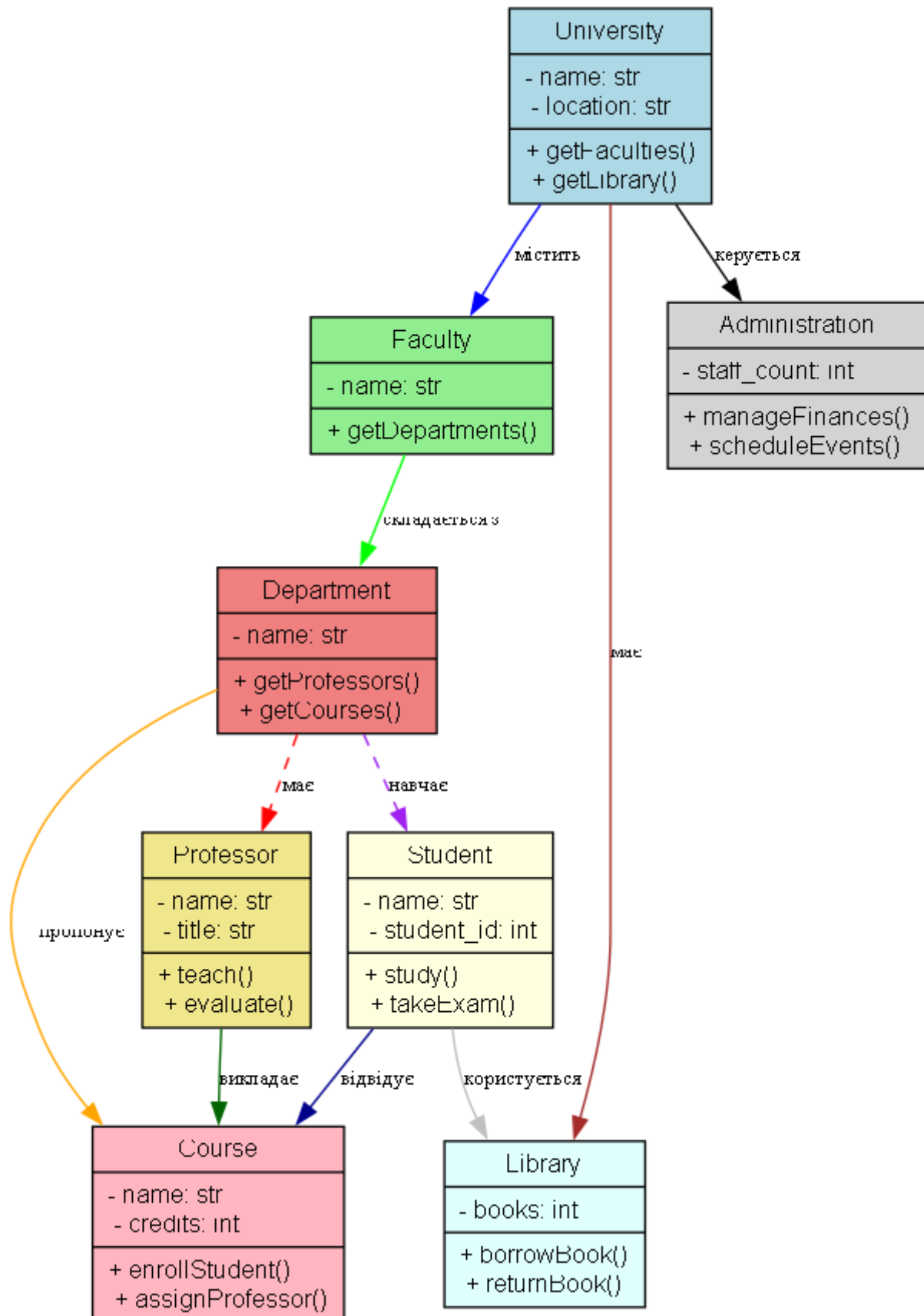
dot.node("Administration", '''{
    Administration |
    - staff_count: int\\l|
    + manageFinances()\\l
    + scheduleEvents()\\l
}''', fillcolor="lightgray", **default_style)

# Додаємо зв'язки між класами
dot.edge("University", "Faculty",
label="містить", color="blue", fontsize="10")
dot.edge("University", "Library", label="має",
color="brown", fontsize="10")
dot.edge("University", "Administration",
label="керується", color="black", fontsize="10")
dot.edge("Faculty", "Department",
label="складається з", color="green",

```

```
fontsize="10")
    dot.edge("Department", "Professor",
label="має", color="red", fontsize="10",
style="dashed")
    dot.edge("Department", "Student",
label="навчає", color="purple", fontsize="10",
style="dashed")
    dot.edge("Department", "Course",
label="пропонує", color="orange", fontsize="10")
    dot.edge("Professor", "Course",
label="викладає", color="darkgreen",
fontsize="10")
    dot.edge("Student", "Course",
label="відвідує", color="darkblue", fontsize="10")
    dot.edge("Student", "Library",
label="користується", color="gray", fontsize="10")
    # Зберігаємо та рендеримо
    dot.render("university_extended_uml",
view=True)

generate_university_uml()
```



## 2. ВИДИ UML-ДІАГРАМ

Unified Modeling Language (UML) — це стандартна мова для візуалізації, специфікації, конструювання та документування програмних систем. Вона

дозволяє описувати як **структуру** системи, так і **поведінку**. UML має кілька важливих особливостей:

У мові UML є 12 типів діаграм:

- 5 типів діаграм описують поведінкові аспекти системи;
- 4 діаграми характеризують статичну структуру додатку;
- 3 діаграми мають назву діаграм реалізації.

***До ключових діаграм належать:***

***1. Діаграма класів (Class Diagram)***

***Призначення:*** показує структуру системи – класи, їх атрибути, методи та зв'язки.

***Приклад:*** система бібліотеки

***2. Діаграма випадків використання (Use Case Diagram)***

***Призначення:*** відображає, які функції система надає користувачам.

***Приклад:*** інтернет-магазин

***3. Діаграма послідовності (Sequence Diagram)***

***Призначення:*** показує, як об'єкти взаємодіють у часі.

***Приклад:*** оформлення замовлення в e-commerce

***4. Діаграма діяльності (Activity Diagram)***

***Призначення:*** описує алгоритм чи бізнес-процес.

***Приклад:*** процес реєстрації користувача

***5. Діаграма станів (State Diagram)***

***Призначення:*** показує життєвий цикл об'єкта.

***Приклад:*** статус замовлення

***6. Діаграма компонентів (Component Diagram)***

**Призначення:** показує фізичні модулі системи та їх залежності.

**Приклад:** веб-додаток

- Компоненти: Frontend (React), Backend (Spring Boot), Database (PostgreSQL)

## **7. Діаграма розгортання (Deployment Diagram)**

**Призначення:** показує, як система розгортається на апаратному забезпеченні.

**Приклад:** розподілений сервіс

# **3. ЗАГАЛЬНА ХАРАКТЕРИСТИКА UML-ДІАГРАМ**

## **1. Стандартність та уніфікованість**

UML є стандартом, прийнятим міжнародною організацією **OMG (Object Management Group)**, що забезпечує уніфікований підхід до проектування систем. Він дозволяє різним командам використовувати одну мову для опису системи, незалежно від її складності та області застосування.

## **2. Візуалізація**

UML використовує графічні елементи для відображення різних аспектів системи. Діаграми є центральною частиною UML і допомагають наочно уявити структуру, поведінку та взаємодії системи.

## **3. Підтримка об'єктно-орієнтованого підходу**

UML широко використовується для об'єктно-орієнтованого аналізу і проектування. Це включає підтримку таких концепцій, як **класи**, **об'єкти**, **наслідування**, **поліморфізм** та **інкапсуляція**. UML дозволяє проектувати складні об'єктно-орієнтовані системи в чіткій і зрозумілій формі.

#### 4. Гнучкість і масштабованість

UML не прив'язана до конкретної мови програмування чи процесу розробки. Вона застосовується для моделювання систем будь-якої складності та в будь-яких галузях, таких як розробка ПЗ, бізнес-моделювання, інженерія систем тощо.

#### 5. Підтримка різних етапів життєвого циклу

UML допомагає моделювати систему на різних етапах її розробки: від початкового аналізу та збору вимог до детального проектування та тестування. UML використовується як для концептуального моделювання (високий рівень абстракції), так і для фізичного (ближче до конкретної реалізації).

#### 6. Стандартизовані нотації

UML має набір чітко визначених графічних елементів для відображення компонентів системи:

- **Прямокутники** для відображення класів або об'єктів
- **Овальні фігури** для випадків використання
- **Стрілки** для відображення взаємодій або відносин між компонентами системи

#### 7. Можливість розширення

UML підтримує механізми розширення, такі як **стереотипи**, **теги** та **обмеження**, що дозволяє адаптувати його до специфічних потреб проєкту або галузі.

#### 8. Інструментарій

Існує багато інструментів для роботи з UML (наприклад, *Enterprise Architect*, *Visual Paradigm*, *Rational Rose*), які дозволяють автоматизувати процес моделювання, генерувати код на основі діаграм або перевіряти коректність моделі.



**Enterprise Architect (EA)** — це фахівець або роль, що відповідає за проєктування та управління комплексною архітектурою підприємства, включаючи його IT-інфраструктуру, бізнес-процеси та технологічні рішення. Основна мета Enterprise Architect полягає у створенні єдиного бачення того, як різні компоненти організації працюють разом, щоб досягти стратегічних цілей компанії. Він допомагає зробити бізнес та IT узгодженими між собою.

#### **Основні завдання Enterprise Architect:**

1. **Визначення архітектури підприємства:** Це означає розуміння поточної структури компанії та проєктування майбутньої моделі, яка дозволить досягти стратегічних бізнес-цілей.
2. **Узгодження бізнесу та IT:** EA забезпечує зв'язок між бізнесом і технологіями, щоб IT-рішення відповідали потребам бізнесу.
3. **Створення архітектурних рамок та стандартів:** Використання різних архітектурних фреймворків, таких як TOGAF або Zachman, для створення структурованої моделі архітектури.
4. **Інтеграція систем:** EA забезпечує, що різні системи, що використовуються в організації, можуть ефективно працювати разом.
5. **Оцінка та вибір технологій:** Визначення, які технології найкраще підходять для поточних і майбутніх потреб компанії.
6. **Контроль змін:** Забезпечення того, що зміни в архітектурі здійснюються відповідно до плану і не викликають збоїв в існуючих процесах.
7. **Робота з зацікавленими сторонами:** Постійна комунікація з бізнес-керівниками, IT-директорами та іншими ключовими фігурами для забезпечення ефективного впровадження стратегій.

#### **9. Моделювання як поведінки, так і структури**

UML дозволяє описувати як **структурні** аспекти системи (класи, компоненти, архітектуру), так і **поведінкові** (взаємодія між компонентами, процеси, потоки даних).

#### **Висновок**

UML є потужним засобом для моделювання складних систем. Його універсальність і багатство інструментів дозволяють створювати як загальні, так

і детальні моделі, що сприяє кращій комунікації між членами команди, зниженню ризиків і покращенню якості кінцевого продукту.

Процес розвитку уніфікованої мови моделювання (unified modeling language – UML) стартував наприкінці XX сторіччя. Однією з цілей UML було забезпечити взаємодію між програмістами шляхом створення загальної, але специфічної мовної конструкції. У перспективі таку структуру можна було б використовувати для розвитку і створення комп'ютерних програм. Взагалі ситуація нагадує історію із програмними моделями типу патернів, які ми розглядали у попередньому підрозділі. Отож, розробники UML створили уніфікований стандарт моделювання, до якого ІТ-фахівці прагнули роками. Тепер, використовуючи UML, ІТ-спеціалісти можуть формалізувати і розуміти складні програмні структури. Інакше кажучи, UML стала об'єднуючою, стандартизованою мовою створення програмних продуктів, що дає можливість моделювати комп'ютерні програми.

UML використовує специфічну діаграмну техніку, що дає змогу зрозуміти принципи створення програм. Розміщенням стандартних діаграм UML у робочому програмному коді створюється реальна можливість полегшити роботу програміста.

#### ***4. ДІАГРАМА КЛАСІВ***

**Class diagram** представлена схематично на рис. 3.9 та являє собою набір загальних атрибутів та операцій.

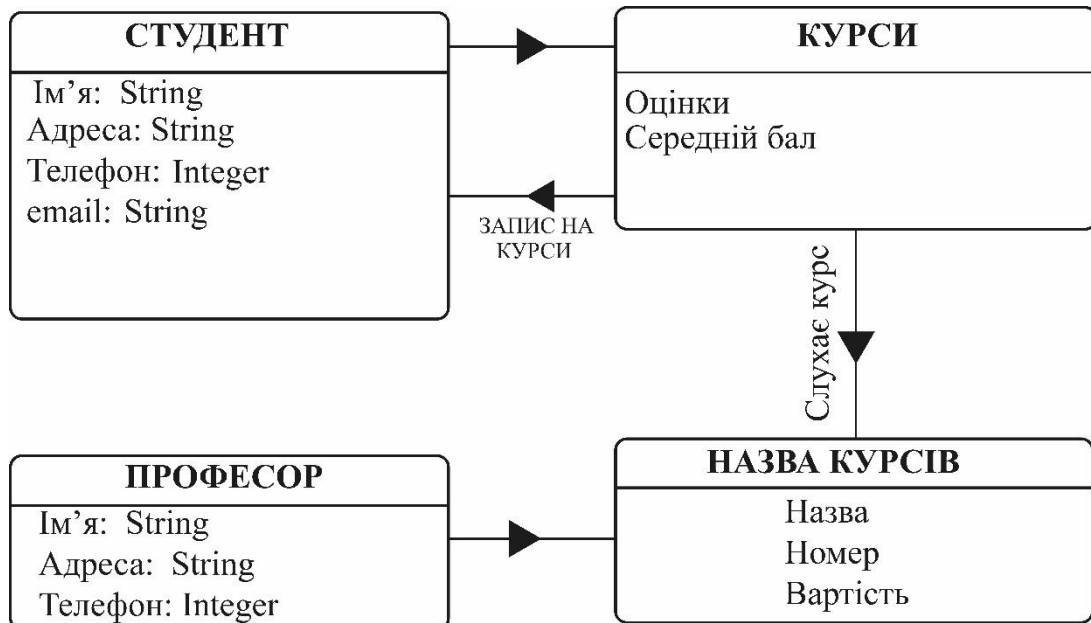


Рис. 3.9. Специфіка діаграми класів

Фактично діаграма класів – це набір статичних елементів моделі. Вона дає найбільш повне і розгорнуте уявлення про зв'язки в програмному коді, функціональність та інформацію про окремі компоненти. Додатки генеруються часто саме з діаграми класів. Для класу **СТУДЕНТ** сформовані записи, що містять атрибути: ім'я, адреса, телефон, email. Схематично зображені зв'язки цієї сутності з іншими: проходженням курсу, який курс слухає, хто професор. У цьому прикладі також представляються функції, які можуть бути застосовані до атрибута **ПРОФЕСОР**.

## [CMM LEC 5-1]

```

from matplotlib import pyplot as plt
from matplotlib import patches
from pathlib import Path

def draw_class(ax, x, y, width, height, title, attributes, methods,
title_font=16, text_font=12):
    rect = patches.FancyBboxPatch((x, y), width, height,
boxstyle="round,pad=0.3", linewidth=1.6)
    ax.add_patch(rect)
    title_height = 0.14 * height
    title_rect = patches.Rectangle((x, y + height - title_height), width,
title_height, linewidth=0)
    ax.add_patch(title_rect)
    sep_y1 = y + height - title_height
    sep_y2 = y + height - title_height - 0.02 - 0.40*height
    ax.plot([x, x+width], [sep_y1, sep_y1], linewidth=1.2)
    ax.plot([x, x+width], [sep_y2, sep_y2], linewidth=1.0)
    ax.text(x + width/2, y + height - title_height/2, title,
fontsize=title_font, fontweight='bold', ha='center', va='center')
    attr_x = x + 0.06 * width
    attr_y_start = sep_y1 - 0.05*height
    line_height = 0.06 * height
    for i, attr in enumerate(attributes):
  
```

```

        ax.text(attr_x, attr_y_start - i*line_height, attr, fontsize=text_font,
ha='left', va='center', family='monospace')
        meth_x = attr_x
        meth_y_start = sep_y2 - 0.05*height
        for i, m in enumerate(methods):
            ax.text(meth_x, meth_y_start - i*line_height, m, fontsize=text_font,
ha='left', va='center', family='monospace')

fig = plt.figure(figsize=(12, 8))
ax = fig.add_subplot(111)
ax.set_xlim(0, 12)
ax.set_ylim(0, 8)
ax.axis('off')

u_x, u_y, u_w, u_h = 0.6, 4.8, 3.2, 2.2
c_x, c_y, c_w, c_h = 6.0, 5.0, 4.0, 2.6
s_x, s_y, s_w, s_h = 6.0, 2.2, 4.0, 2.2

user_attrs = ["- id: Long", "- name: String", "- email: String"]
user_methods = []

controller_methods = [
    "+ getAllUsers(): List<User>",
    "+ getUserById(id: Long): ResponseEntity<User>",
    "+ createUser(user: User): ResponseEntity<User>",
    "+ deleteUser(id: Long): ResponseEntity<Void>"
]

service_methods = [
    "+ getAllUsers(): List<User>",
    "+ getUserById(id: Long): Optional<User>",
    "+ saveUser(user: User): User",
    "+ deleteUser(id: Long): void"
]

draw_class(ax, u_x, u_y, u_w, u_h, "User", user_attrs, user_methods,
title_font=18, text_font=12)
draw_class(ax, c_x, c_y, c_w, c_h, "UserController", [], controller_methods,
title_font=18, text_font=11)
draw_class(ax, s_x, s_y, s_w, s_h, "UserService", [], service_methods,
title_font=18, text_font=11)

ux_line_y = u_y + u_h/2
cx_line_y = c_y + c_h/2
ax.plot([u_x + u_w, c_x], [ux_line_y, cx_line_y], linewidth=1.4)
sx_line_y = s_y + s_h/2
ax.plot([u_x + u_w, s_x], [ux_line_y, sx_line_y], linewidth=1.4)

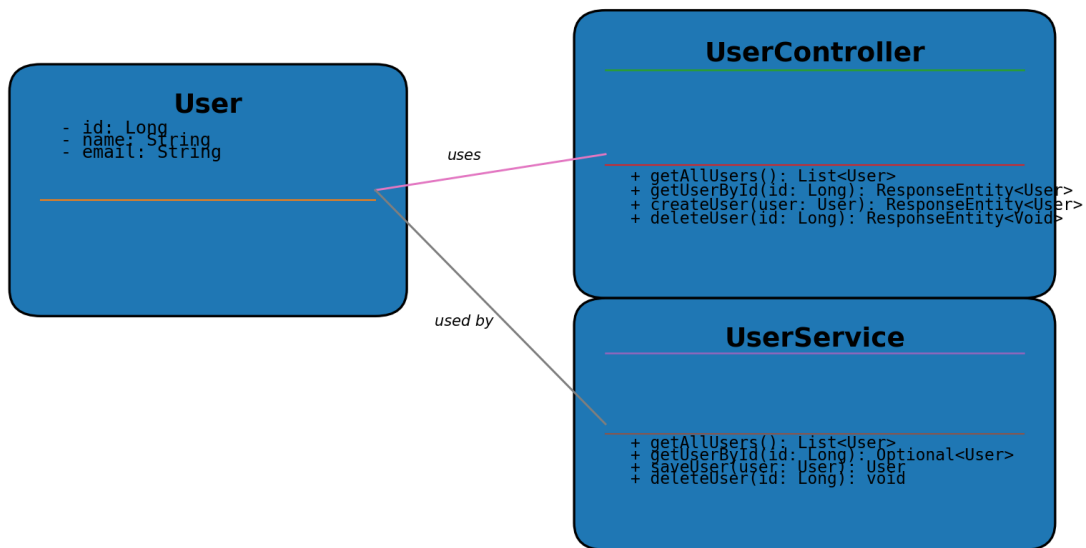
ax.text((u_x+u_w + c_x)/2 - 0.25, (ux_line_y + cx_line_y)/2 + 0.18, "uses",
fontsize=10, ha='center', va='center', style='italic')
ax.text((u_x+u_w + s_x)/2 - 0.25, (ux_line_y + sx_line_y)/2 - 0.16, "used by",
fontsize=10, ha='center', va='center', style='italic')

fig.suptitle("UML Class Diagram - RESTful User API (expanded graphic)",
fontsize=20, y=0.95, fontweight='bold')
plt.tight_layout(rect=[0, 0, 1, 0.93])

out_dir = Path(".")
plt.savefig(out_dir / "uml_class_diagram_generated.png", dpi=150)
print("Saved uml_class_diagram_generated.png")

```

## UML Class Diagram — RESTful User API (expanded graphic)



### РОЗШИФРУВАННЯ ДІАГРАМИ:

- **+getAllusers ()** → отримати список користувачів
  - **+getUserById ()** → отримати користувача за ID
  - **+createUser ()** → створити нового користувача
  - **+deleteUser ()** → видалити користувача
- 
- **+getAllusers ()** → отримати всіх користувачів з БД
  - **+getUserById ()** → перевірити, чи існує користувач з таким email
  - **saveUser ()** → додати нового користувача
  - **deleteUser ()** → видалити користувача

## 5. ДІАГРАМА АКТИВНОСТІ

**Activity diagram** досить часто використовується на практиці (рис. 3.10). Діаграма подібного типу описує динамічні аспекти поведінки системи,

використовуючи блок-схеми, що наочно відображають ті чи інші процеси та структуру логіки процедур. Взагалі, йдеться про процедуру алгоритму дій щодо взаємодії елементів у межах системи або взаємодії між кількома системами. Приклад такої діаграми доцільно навести для інтернет-магазину. У цьому випадку діаграма Activity diagram для сайту магазину максимально доступно пояснює, які функціонують координаційні зв'язки у системі. Рушійним елементом системи у цьому випадку є **Actor** (у якості такого об'єкта у нашому випадку виступає покупець), що відкрив сайт та здійснює процедуру замовлення (рис. 3.10). Далі відбувається розгалуження, а саме: перевіряється, чи є **Actor** оптовиком («Так» або «Ні»). Якщо «Ні», то замовлення відправляється в **retail.CRM**. Дано пояснення наведеному вище запису. Слово *retail*

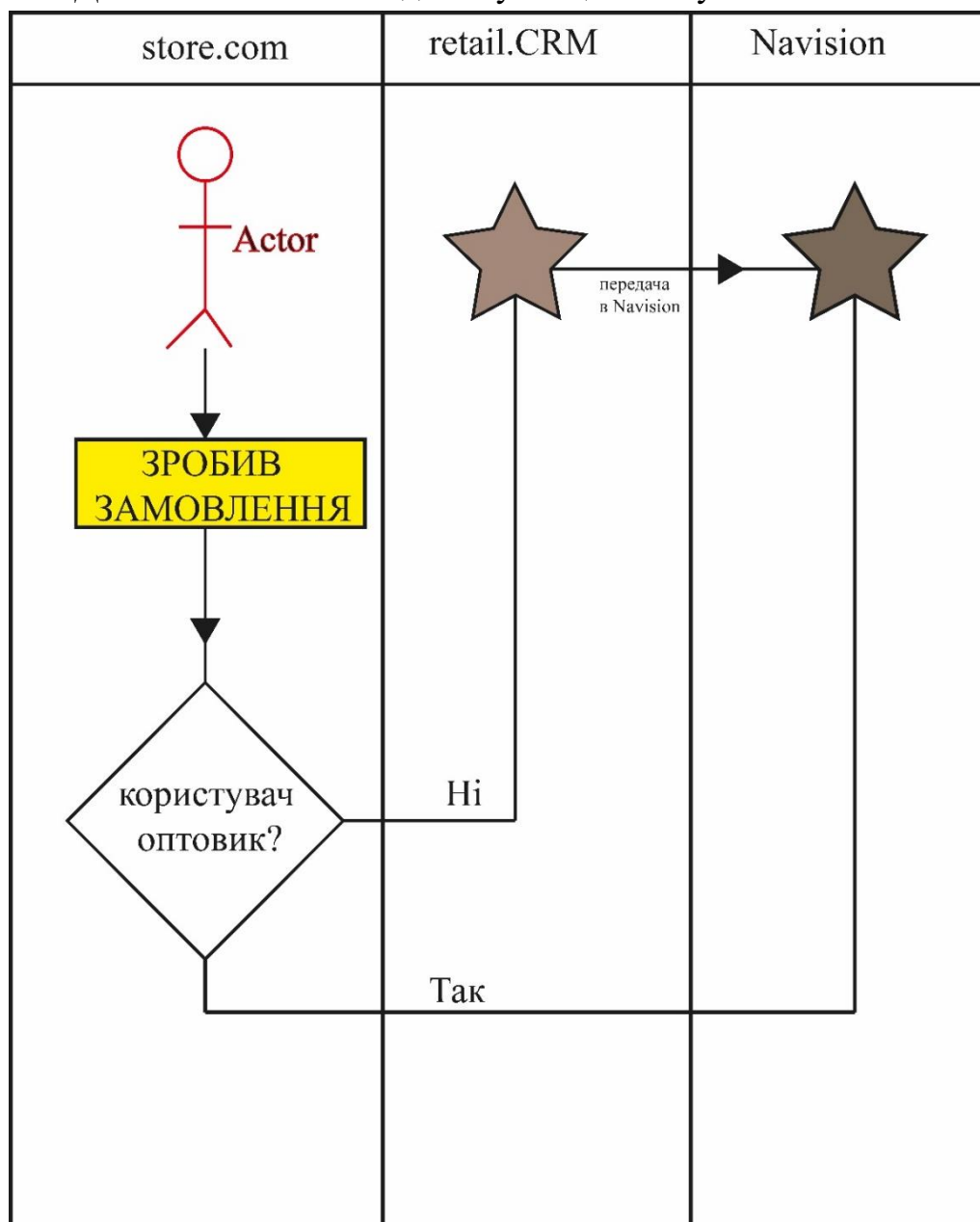


Рис. 3.10. Activity diagram (діаграма активностей), спроектована для інтернет-магазину

перекладається як «роздрібна торгівля». Визначення **CRM** розшифровується як **Customer Relationship Management**, що означає «управління відносинами з

клієнтами» і стосується всіх стратегій, методів, інструментів і технологій, які використовує бізнес для розвитку, утримання і залучення клієнтів. **Customer Relationship Management** – це особливий підхід до ведення бізнесу, за якого на перше місце діяльності компанії ставиться клієнт. Основна мета впровадження CRM-стратегії – створення єдиної системи з залучення нових і розвитку наявних клієнтів. Управляти відносинами означає залучати нових клієнтів, а нейтральних покупців перетворювати в лояльних клієнтів і з постійних клієнтів формувати бізнес-партнерів.

Продовжимо аналіз логіки згідно з рис. 3.10. Якщо логічний блок видає «Так», то його замовлення потрапляє в **Navision** – програма для управління підприємством малого та середнього розміру. Зокрема, до складу Microsoft Dynamics NAV 2016 входять додаткові оптимізовані програми для пристроїв усіх форм-факторів, які працюють під керуванням Android, iOS або Windows. Контроль бізнес-процесів реалізується за допомогою конфігурованих інформаційних панелей Microsoft Power BI, що забезпечують миттєвий доступ до ключових показників ефективності в інтуїтивному середовищі, призначеному для безпечної групової роботи. Реалізовано також інтеграцію з Microsoft Dynamics CRM Online, що дає змогу реалізувати можливості збільшення продажів, покращення обслуговування клієнтів та оптимізації ланцюжка операцій від замовлення до отримання оплати. На рис. 3.10 між **retail.CRM** і **Navision** зображено взаємодію між цими програмними компонентами, зокрема з метою здійснення процедури синхронізації залишків.

Звичайно, що подібні діаграми можна і потрібно доповнювати та розширювати. Такі діаграми стають важливим складником документації і створюють загальне уявлення про роботу системи в цілому.

## 6. ДІАГРАМА ВИПАДКІВ ВИКОРИСТАННЯ (USE CASE DIAGRAM)

**Use-case diagram** (рис. 3.10) містить два важливих атрибуту:

1) **Actor** (учасник) – спектр логічно пов'язаних ролей, виконуваних під час взаємодії з прецедентами або сутностями. Учасником (**Actor**) може бути система загалом, людина, підсистема, клас або будь-яка інша сутність;

2) **Use-case** (прецедент) – опис окремого аспекту поведінки системи з погляду користувача. Прецедент не показує, як досягається певний результат, а тільки показує, що саме виконується.

Розберемо класичний приклад студента у бібліотеці, де є два учасники: бібліотекар і студент (рис. 3.11). *Прецеденти для студента*: пошук книг у

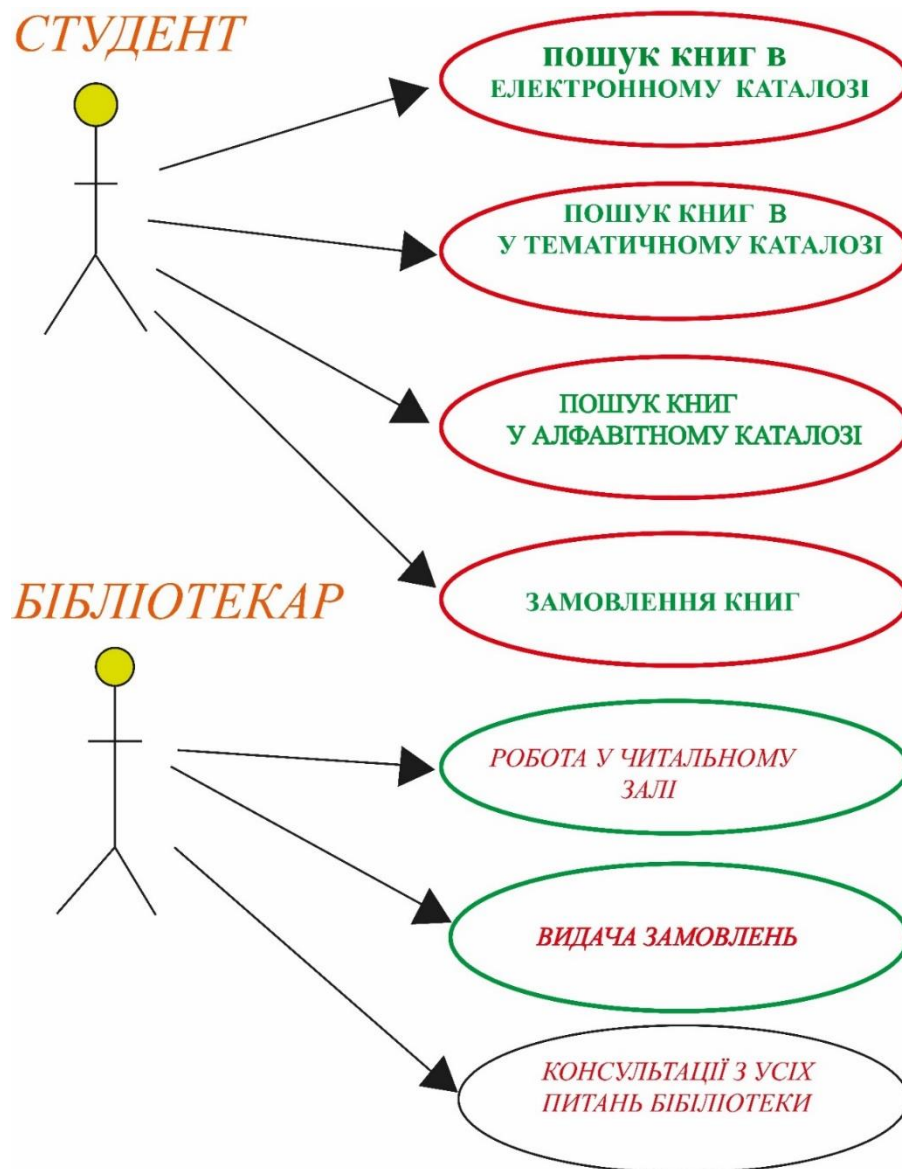


Рис. 3.11. Типовий варіант Use-case diagram

електронному каталозі з використанням комп'ютерів зі спеціальним програмним забезпеченням та інстальованою базою даних, пошук книг у тематичному каталозі, пошук книг у алфавітному каталозі і, нарешті, замовлення книг чи іншої навчальної або художньої літератури.

*Прецеденти для бібліотекаря:* робота у читальному залі у плані організації роботи читального залу загалом, видача замовлення студентам, консультації з усіх питань бібліотечного обслуговування (консультації щодо використання електронної пошукової системи, рекомендації книг із теми, заповнення бланків замовлення та ін.).

Отже, Use-case-діаграми *спрямовані допомогти командам розробників візуалізувати функціональні вимоги системи*. Такі діаграми зазвичай показують спектр варіантів використання – або всі випадки використання для всієї системи,



або лише для окремої групи. Щоб показати варіант використання на Use-case діаграмі, потрібно намалювати овал посередині діаграми та ввести назву варіанта використання у центрі овалу або під ним (рис. 3.11).

Потрібно також застосовувати просту та зрозумілу діаграмну техніку для зображення взаємозв'язків між «акторами» та варіантами використання.

*Use-case* діаграма зазвичай використовується для того, щоб пов'язати функції високого рівня системи з областю застосування системи. Подивившись на діаграму Use-case (рис. 3.11), можна *легко визначити функції, які розглядувана система може виконувати*. Зокрема, ця система дає змогу бібліотекарю вести статистику відвідування студентами читального залу та статистику використання книг.

## 7. ДІАГРАМА ПОСЛІДОВНОСТІ (SEQUENCE DIAGRAM)

*Sequence Diagram* (діаграма послідовності) використовується для уточнення діаграм прецедентів та описує деякі поведінкові аспекти системи. *Така діаграма відображає взаємодію об'єктів у динамічному режимі, інакше – у часі*. Водночас інформація набуває вигляду повідомлень, а взаємодія об'єктів передбачає обмін цими повідомленнями.

### [CMM LEC 5-3]

```
import matplotlib.pyplot as plt

actors = ["Client", "UserController",
"UserService", "UserRepository", "Database"]
x_positions = [1, 3.5, 6, 8.5, 11]

fig, ax = plt.subplots(figsize=(12, 6))
ax.axis('off')
ax.set_xlim(0, 12)
ax.set_ylim(0, 8)

for x, actor in zip(x_positions, actors):
    ax.text(x, 7.7, actor, ha='center',
va='center', fontsize=12, fontweight='bold')
    ax.plot([x, x], [7.4, 0.5], linestyle="--",
color="gray")

messages = [
    ("send POST /users (JSON)", 7.0, 1, 2),
```

```

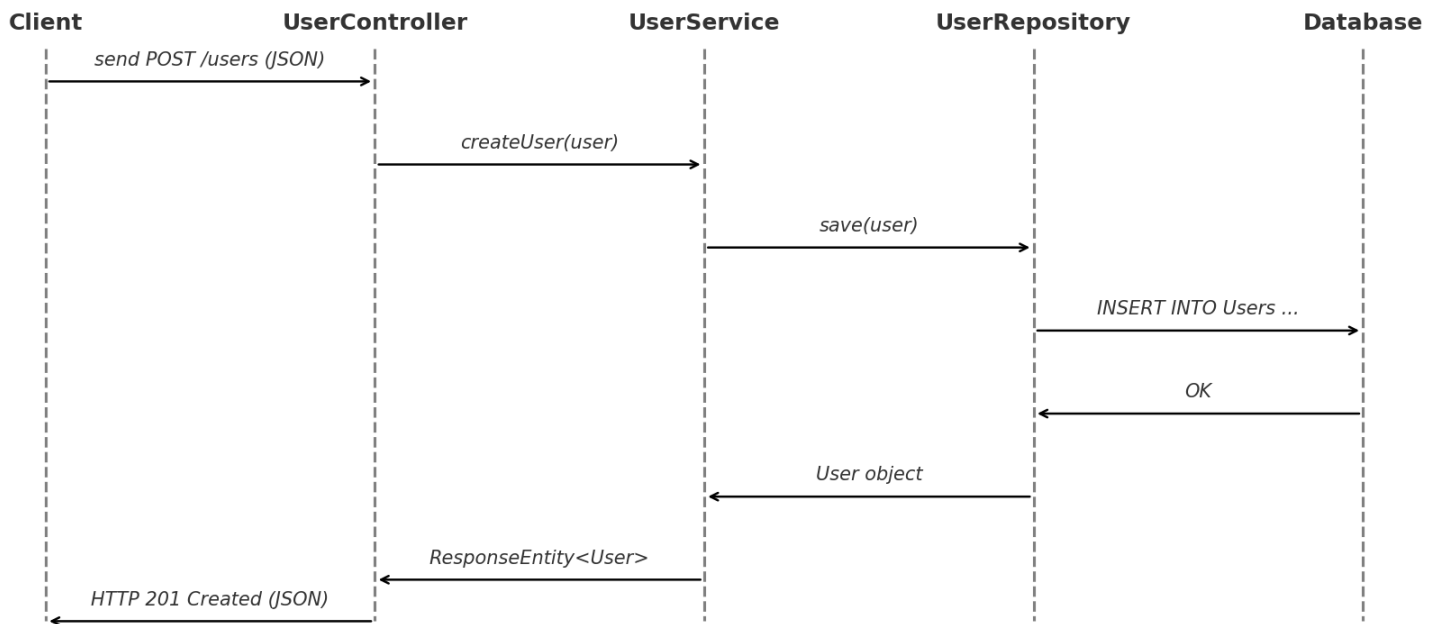
        ("createUser(user)", 6.0, 2, 3),
        ("save(user)", 5.0, 3, 4),
        ("INSERT INTO Users ...", 4.0, 4, 5),
        ("OK", 3.0, 5, 4),
        ("User object", 2.0, 4, 3),
        ("ResponseEntity<User>", 1.0, 3, 2),
        ("HTTP 201 Created (JSON)", 0.5, 2, 1)
    ]

for msg, y, src, tgt in messages:
    x1, x2 = x_positions[src-1], x_positions[tgt-
1]
    ax.annotate("", xy=(x2, y), xytext=(x1, y),
arrowprops=dict(arrowstyle="->", lw=1.2))
    ax.text((x1 + x2)/2, y+0.15, msg, ha='center',
va='bottom', fontsize=10, style="italic")

fig.suptitle("UML Sequence Diagram – Create User",
fontsize=16, fontweight='bold')
plt.tight_layout(rect=[0, 0, 1, 0.93])
plt.savefig("uml_sequence_diagram.png", dpi=150)
print("Saved uml_sequence_diagram.png")

```

## UML Sequence Diagram — Create User



### Учасники (Actors)

1. **Client** – будь-який споживач API (браузер, мобільний застосунок, інший сервіс).
2. **UserController** – контролер Spring Boot, який приймає HTTP-запити.
3. **UserService** – сервісний шар, що містить бізнес-логіку створення користувача.
4. **UserRepository** – шар доступу до даних (DAO/Repository), який взаємодіє з базою даних.
5. **Database** – власне сховище даних (SQL/NoSQL).

### Послідовність взаємодії

1. **Client** → **UserController**: відправляє запит  
*POST /users з JSON-даними нового користувача.*
2. **UserController** → **UserService**: виклик методу  
*createUser(user) – контролер передає об'єкт користувача в сервіс.*
3. **UserService** → **UserRepository**: збереження  
*save(user) – сервіс доручає репозиторію записати дані.*
4. **UserRepository** → **Database**: SQL-запит

*INSERT INTO Users ... – у базу записується новий користувач.*

5. **Database** → **UserRepository**: підтвердження

*OK – база даних підтверджує успішне збереження.*

6. **UserRepository** → **UserService**: повертає створений об'єкт

*User object (менер із згенерованим ID).*

7. **UserService** → **UserController**: обгортає у відповідь

*ResponseEntity<User> – повертає готовий об'єкт з HTTP-статусом.*

8. **UserController** → **Client**: фінальна відповідь

*HTTP 201 Created (JSON) – клієнт отримує підтвердження і створений об'єкт користувача.*

### **Ключові ідеї**

- **Контролер** лише приймає та відправляє дані (не містить логіки).
- **Сервіс** виконує основну бізнес-логіку.
- **Репозиторій** відповідає тільки за доступ до БД.
- **База даних** зберігає стан системи.

### **8. ДІАГРАМА РОЗГОРТАННЯ (Deployment Diagram)**

*Deployment Diagram* (діаграма розгортання) відображає графічне представлення інфраструктури, для якої буде застосовано додаток, а саме: структуру системи загалом та розподіл її компонентів по вузлах. Зображуються також з'єднання – маршрути передачі даних між вузлами. Діаграма допомагає ефективніше організувати компоненти, від чого залежить продуктивність системи, а також вирішити допоміжні проблеми, зокрема пов'язані з безпекою.

Як бачимо, на перший погляд банальний набір фігур і стрілок може значно спростити вирішення складних завдань у програмуванні, допомогти під час вибору оптимального рішення і під час розробки технічної документації. Можемо зробити такі висновки:

- будувати діаграми нескладно;
- діаграми легко зчитуються та зрозуміло спроектовані;
- діаграми – чудовий інструмент для реалізації архітектури системи загалом;
- необхідні для документування будь-якої нетривіальної системи та дають змогу зрозуміти зв'язки між її модулями.

## [CMM LEC 5-2]

**UML (Unified Modeling Language)**, який використовується для візуалізації, специфікації, проектування та документування компонентів програмних систем. У контексті Python, можна створити програму, яка моделює просту UML-діаграму за допомогою графічних бібліотек, таких як Tkinter або Graphviz. Ми можемо реалізувати, наприклад, діаграму класів або об'єктів.

Ось приклад Python-програми для створення простої UML-діаграми класів з графічним інтерфейсом за допомогою Tkinter:

```
import tkinter as tk
from tkinter import simpledialog

class UMLDiagram:
    def __init__(self, root):
        self.root = root
        self.root.title("UML Class Diagram")

        self.canvas = tk.Canvas(root, width=600, height=400, bg="white")
        self.canvas.pack()

        # Створюємо меню для додавання класів та зв'язків
        menu = tk.Menu(root)
        root.config(menu=menu)

        add_menu = tk.Menu(menu)
        menu.add_cascade(label="Add", menu=add_menu)
        add_menu.add_command(label="Add Class", command=self.add_class)
        add_menu.add_command(label="Add Relationship",
command=self.add_relationship)

        self.classes = []
        self.relationships = []

    def add_class(self):
        # Запитуємо у користувача ім'я класу
        class_name = simpledialog.askstring("Input", "Enter class name:")
```

```

        if class_name:
            # Малюємо простий прямокутник для класу
            x, y = 100 + len(self.classes) * 100, 100
            class_box = self.canvas.create_rectangle(x, y, x + 80, y + 50,
fill="lightblue")
            class_text = self.canvas.create_text(x + 40, y + 25,
text=class_name)
            self.classes.append((class_name, class_box, class_text))

    def add_relationship(self):
        if len(self.classes) < 2:
            tk.messagebox.showinfo("Error", "At least 2 classes are required to
add a relationship.")
            return

        # Запитуємо у користувача дві назви класів для створення зв'язку
        class1 = simplifiedialog.askstring("Input", "Enter the first class name:")
        class2 = simplifiedialog.askstring("Input", "Enter the second class name:")

        class1_box, class2_box = None, None

        for class_name, box, text in self.classes:
            if class_name == class1:
                class1_box = box
            if class_name == class2:
                class2_box = box

        if class1_box and class2_box:
            x1, y1, x2, y2 = self.canvas.coords(class1_box)
            x3, y3, x4, y4 = self.canvas.coords(class2_box)
            # Створюємо лінію для з'єднання класів
            self.canvas.create_line(x2, (y1 + y2) // 2, x3, (y3 + y4) // 2,
arrow=tk.LAST)

if __name__ == "__main__":
    root = tk.Tk()
    app = UMLDiagram(root)
    root.mainloop()

```

## Опис програми:

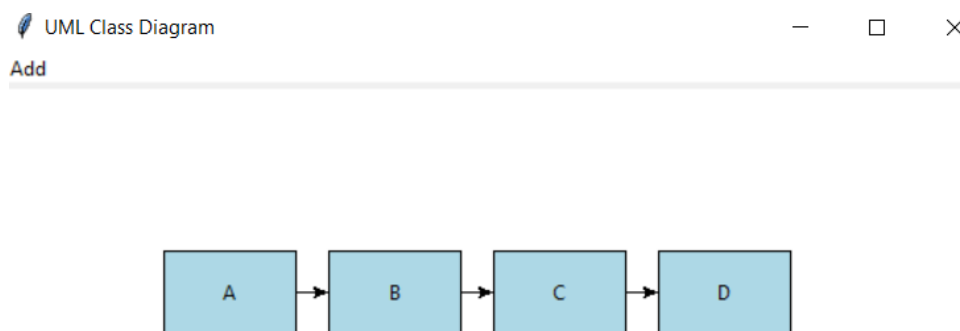
1. **UMLDiagram** — основний клас, що відповідає за відображення UML-діаграми класів.
2. **canvas** — область для малювання, де будуть розташовані класи та зв'язки.
3. **add\_class()** — функція для додавання нового класу на діаграму у вигляді прямокутника з текстом.
4. **add\_relationship()** — функція для додавання зв'язків між класами за допомогою ліній.

## Як працює програма:

1. Після запуску відкривається графічне вікно з меню.

2. Через меню "Add" користувач може додати новий клас, ввівши його ім'я.
3. Кожен клас відображається у вигляді прямокутника.
4. Користувач може також додати зв'язки між класами, ввівши імена двох класів, між якими потрібно створити зв'язок.

Ця програма — базовий приклад побудови UML-діаграми класів. Її можна розширити, додавши більше елементів UML, таких як атрибути та методи для класів, інші типи зв'язків (наприклад, агрегація, наслідування), і більш складну графіку для точного розташування елементів.



## ***9. АНАЛІЗ ЛІТЕРАТУРНИХ ДЖЕРЕЛ***

**1. IEEE Transactions on Software Engineering**

**2. Empirical Software Engineering**

**3. Software Quality Journal**

***4. Innovations in Systems and Software Engineering***

**5. International Journal of System Assurance Engineering and Management**

**6. Journal of Systems and Software**

## ***10. UML та AI***

- **Генерація UML-діаграм**

За допомогою алгоритмів NLP (Natural Language Processing) можна автоматично створювати UML-діаграми з текстових специфікацій вимог.

- **Оптимізація моделей**

AI може підказувати найкращі архітектурні рішення, знаходити дублювання або помилки у діаграмах.

- **Автоматична валідація**

Машинне навчання здатне виявляти нетипові або помилкові сценарії у моделях UML, що важливо для великих проєктів.

- **UML + AI**

UML у поєднанні з AI використовується для автоматичної генерації коду з діаграм (Model-to-Code).

## Приклади застосування UML + AI

- **Розробка систем для «розумних міст»:** UML описує архітектуру транспортних систем, а AI аналізує дані сенсорів і оптимізує маршрути.
- **Медичні інформаційні системи:** UML застосовується для проектування процесів обробки даних пацієнтів, а AI — для діагностики.
- **Промисловість 4.0:** UML моделює виробничі процеси, а AI оптимізує використання ресурсів.

## [CMM LEC 5-3]

```
# Demo: UML ↔ AI interaction in Python (with  
Graphviz rendering)  
# This script demonstrates how UML (Unified  
Modeling Language) and AI (machine learning) can  
interact.
```

```
from dataclasses import dataclass, field  
from typing import List, Dict, Tuple  
import re  
import random
```



```

import json

# --- 1) Example requirements (natural language) ---
requirements = """
Smart Traffic Light System for urban
intersections:
- Each Intersection has multiple TrafficLights and
Sensors (vehicle sensors, pedestrian sensors).
- A TrafficLight has states: RED, GREEN, YELLOW
and an associated Controller.
- Sensors stream vehicle_count and
pedestrian_count metrics to the Controller.
- The Controller aggregates sensor data, runs a ML
model to predict congestion_level and
  decides signal_timing (in seconds) for each
TrafficLight.
- The system stores historical_data for learning
and offline model retraining.
- Alerts are generated when sensor data is missing
or abnormal.
"""

print("=== Requirements (input) ===")
print(requirements.strip())

# --- 2) Very simple "AI" extractor (heuristic /
keyword-based) ---
keywords_to_classes = [
    ("Intersection", ["id", "location",
"traffic_lights"]),
    ("TrafficLight", ["id", "state",
"controller_id"]),
    ("Sensor", ["id", "type", "vehicle_count",
"pedestrian_count"]),
    ("Controller", ["id", "managed_lights",
"model_version"]),
    ("MLModel", ["model_id", "model_type",
"congestion_prediction"]),
    ("HistoricalData", ["timestamp",
"sensor_snapshot", "congestion_level"]),

```

```

        ("Alert", ["id", "timestamp", "message",
"severity"]))
]

```

```

detected = []
for name, attrs in keywords_to_classes:
    if re.search(r"\\b" + re.escape(name.lower()))
+ r"\\b", requirements.lower()) or
re.search(r"\\b" + re.escape(name) + r"\\b",
requirements):
    detected.append((name, attrs))

```

```

print("\n=== Detected classes (via heuristic
extractor) ===")
for name, attrs in detected:
    print(f"- {name}: {attrs}")

```

*# --- 3) UML-like model representation ---*

```

@dataclass

```

```

class UMLClass:
    name: str
    attributes: List[str] =
field(default_factory=list)
    methods: List[str] =
field(default_factory=list)

```

```

@dataclass

```

```

class UMLModel:
    classes: Dict[str, UMLClass] =
field(default_factory=dict)
    relations: List[Tuple[str, str, str]] =
field(default_factory=list)

```

```

    def add_class(self, cls: UMLClass):
        self.classes[cls.name] = cls

```

```

    def add_relation(self, a, b, rel):
        self.relations.append((a, b, rel))

```

```

uml = UMLModel()

```

```

# Add detected classes
for name, attrs in detected:
    methods = []
    if name == "Controller":
        methods = ["aggregate_sensors()",
"predict_congestion()", "set_signal_timing()"]
    if name == "MLModel":
        methods = ["train()", "predict()"]
    uml.add_class(UMLClass(name=name,
attributes=attrs, methods=methods))

# Add relations (heuristic)
uml.add_relation("Intersection", "TrafficLight",
"1..* contains")
uml.add_relation("TrafficLight", "Controller",
"managed_by")
uml.add_relation("Sensor", "Controller",
"streams_to")
uml.add_relation("Controller", "MLModel", "uses")
uml.add_relation("Controller", "HistoricalData",
"stores_to")
uml.add_relation("Controller", "Alert",
"generates")

# --- 4) Render UML diagram with Graphviz ---
def render_uml_with_graphviz(uml_model,
filename="uml_diagram"):
    try:
        from graphviz import Digraph
    except ImportError:
        print("Graphviz not installed. Please
install with: pip install graphviz")
        return None

    dot = Digraph(comment='UML Class Diagram',
format='png')
    dot.attr('node', shape='record')

    for cls in uml_model.classes.values():
        label = "{" + cls.name + "|" +
"\l".join(cls.attributes) + "\l|" +

```

```

"\l".join(cls.methods) + "\l}"
    dot.node(cls.name, label=label)

    for a,b,rel in uml_model.relations:
        dot.edge(a, b, label=rel)

    outpath = dot.render(filename, cleanup=True)
    return outpath

path = render_uml_with_graphviz(uml,
filename="uml_diagram_demo")
if path:
    print(f"\nUML diagram rendered and saved to
{path}")

# --- 5) UML → AI feature extraction and ML
training ---
def extract_features_from_uml(uml_model):
    num_classes = len(uml_model.classes)
    total_attrs = sum(len(c.attributes) for c in
uml_model.classes.values())
    has_ml = 1 if "MLModel" in uml_model.classes
else 0
    controller_methods =
len(uml_model.classes.get("Controller",
UMLClass("Controller")).methods)
    rel_count = len(uml_model.relations)
    return [num_classes, total_attrs, has_ml,
controller_methods, rel_count]

base_features = extract_features_from_uml(uml)
X = []
y = []
for i in range(100):
    perturb = [max(0, int(v + random.choice([-
1,0,1])*random.random()*2)) if isinstance(v,int)
else v for v in base_features]
    label = 1 if (perturb[2]==1 and perturb[1] >
8) else 0
    if random.random() < 0.05:
        label = 1 - label

```

```

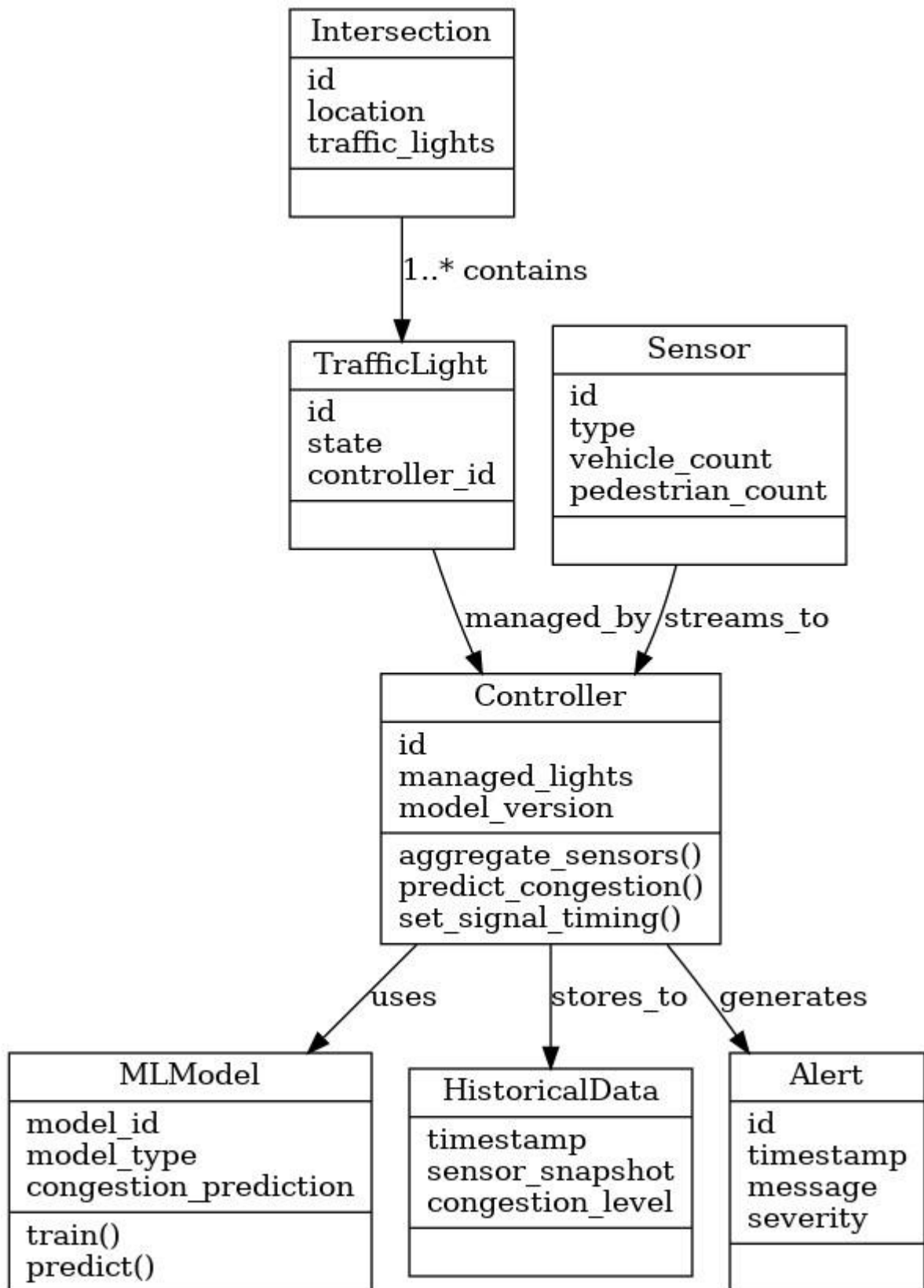
        X.append(perturb)
        y.append(label)

try:
    from sklearn.model_selection import
train_test_split
    from sklearn.linear_model import
LogisticRegression
    from sklearn.metrics import accuracy_score

    X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size=0.2,
random_state=42)
    clf = LogisticRegression(max_iter=200)
    clf.fit(X_train, y_train)
    y_pred = clf.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    print("\nTrained ML model accuracy:", acc)
except Exception as e:
    print("\nML training skipped (scikit-learn
not installed). Error:", str(e))

# Save UML model as JSON
uml_serializable = {
    "classes": {name: {"attributes":
cls.attributes, "methods": cls.methods} for name,
cls in uml.classes.items()},
    "relations": uml.relations
}
with open("uml_model.json", "w", encoding="utf-8")
as f:
    json.dump(uml_serializable, f, indent=2,
ensure_ascii=False)
print("\nSaved UML model to uml_model.json")

```



Діаграма, згенерована з використанням ІІІ